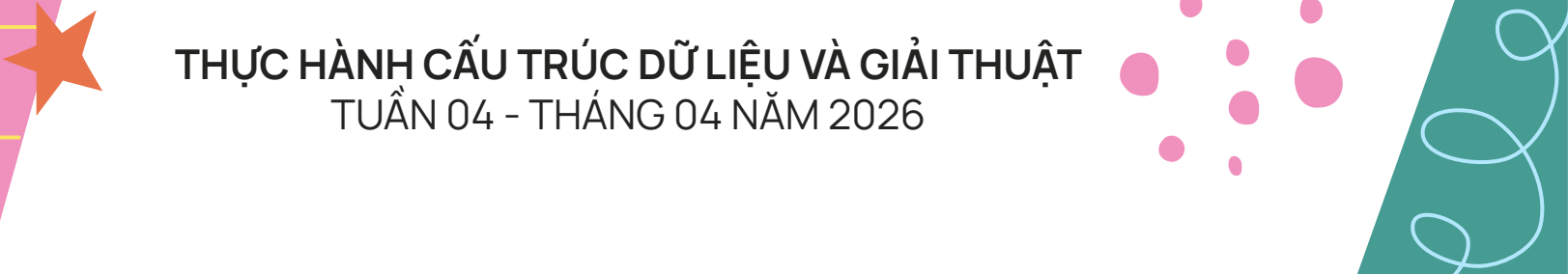




TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN - ĐHQG TP HCM
KHOA TOÁN - TIN HỌC
BỘ MÔN ỨNG DỤNG TIN HỌC



BUBBLE SORT



THỰC HÀNH CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT
TUẦN 04 - THÁNG 04 NĂM 2026

NỘI DUNG

01 GIỚI THIỆU

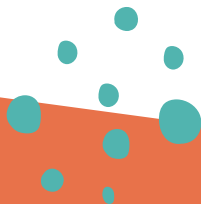
BUBBLE SORT

02 BÀI TẬP TẠI LỚP

Algorithm building
Counting swaps and comps

03 BÀI TẬP VỀ NHÀ

Scenarios



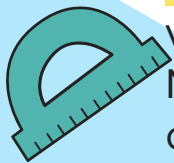
1. GIỚI THIỆU



Thuật toán **Bubble sort** là một thuật toán sắp xếp đơn giản nhưng không hiệu quả cao, được sử dụng để sắp xếp một danh sách các phần tử theo thứ tự tăng dần hoặc giảm dần.

Bubble sort hoạt động bằng cách so sánh các phần tử liền kề trong danh sách và hoán đổi chúng nếu chúng không theo thứ tự mong muốn. Quá trình này được lặp lại cho đến khi không còn phần tử nào cần hoán đổi, tức là danh sách đã được sắp xếp.

Bubble sort thường được sử dụng để giới thiệu các khái niệm cơ bản về thuật toán sắp xếp và làm quen với việc thao tác trên mảng. Ngoài ra, nếu danh sách đầu vào đã gần như sắp xếp, **Bubble sort** có thể là thuật toán hiệu quả hơn so với các thuật toán sắp xếp khác.

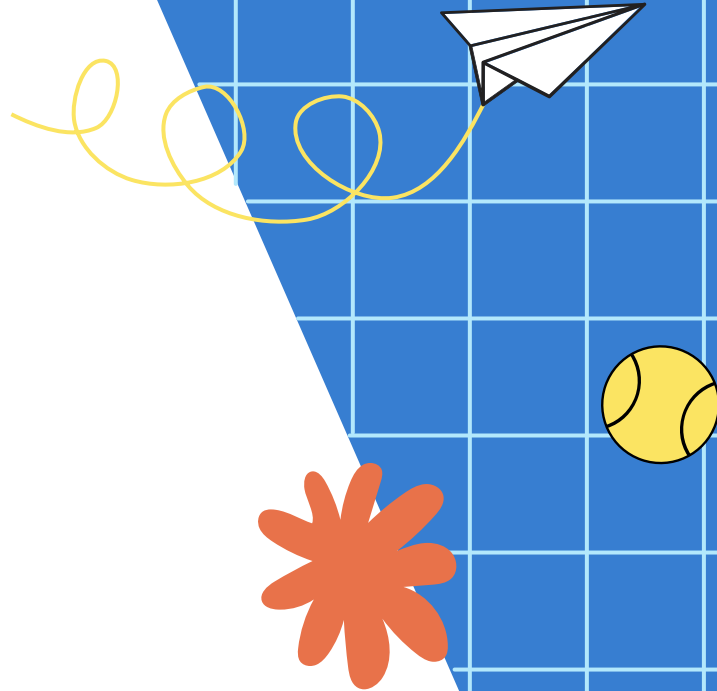


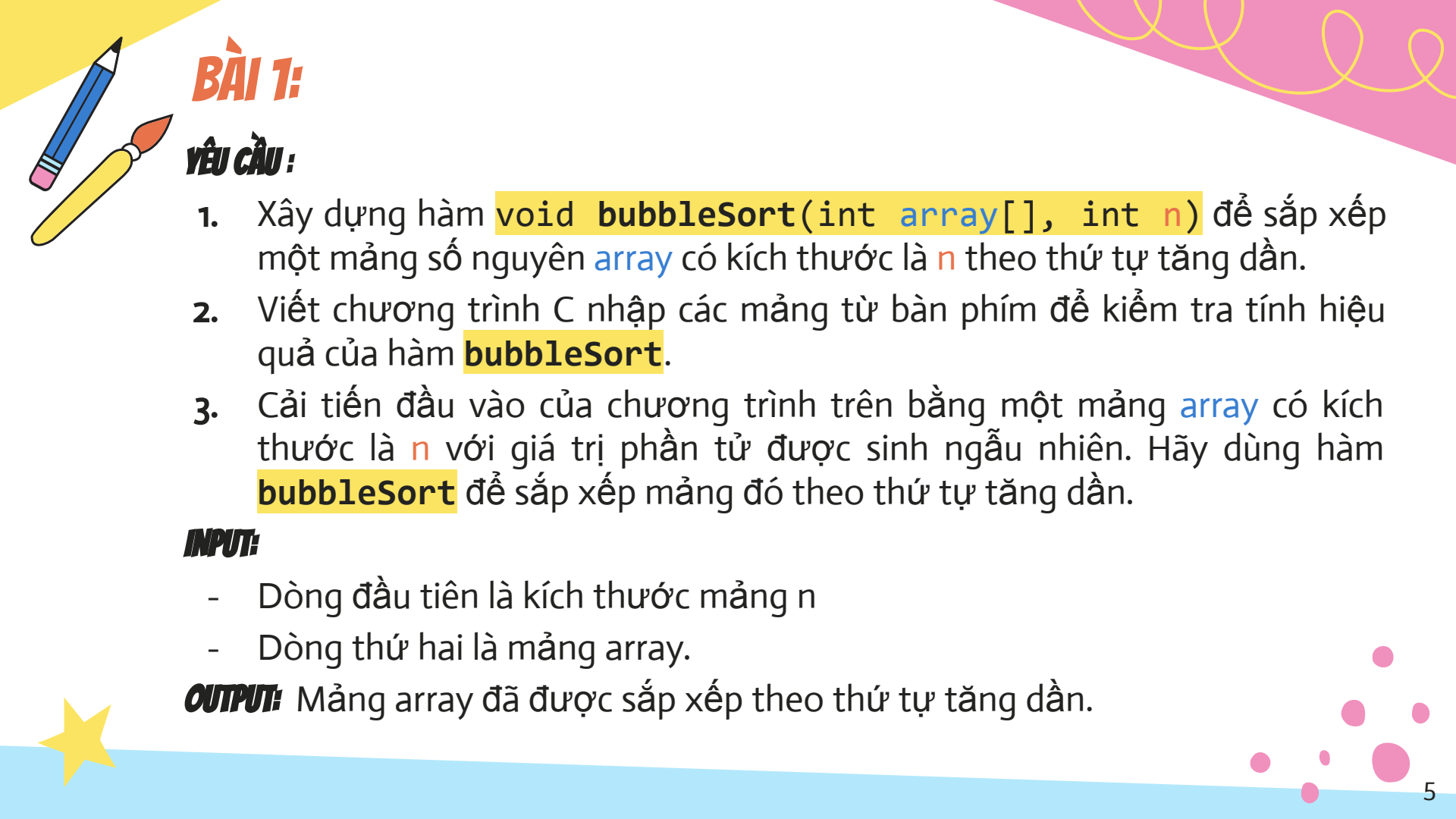


02

BÀI TẬP **TẠI LỚP**

Làm quen với thuật toán sắp xếp
mảng cơ bản nhất





BÀI 1:

YÊU CẦU:

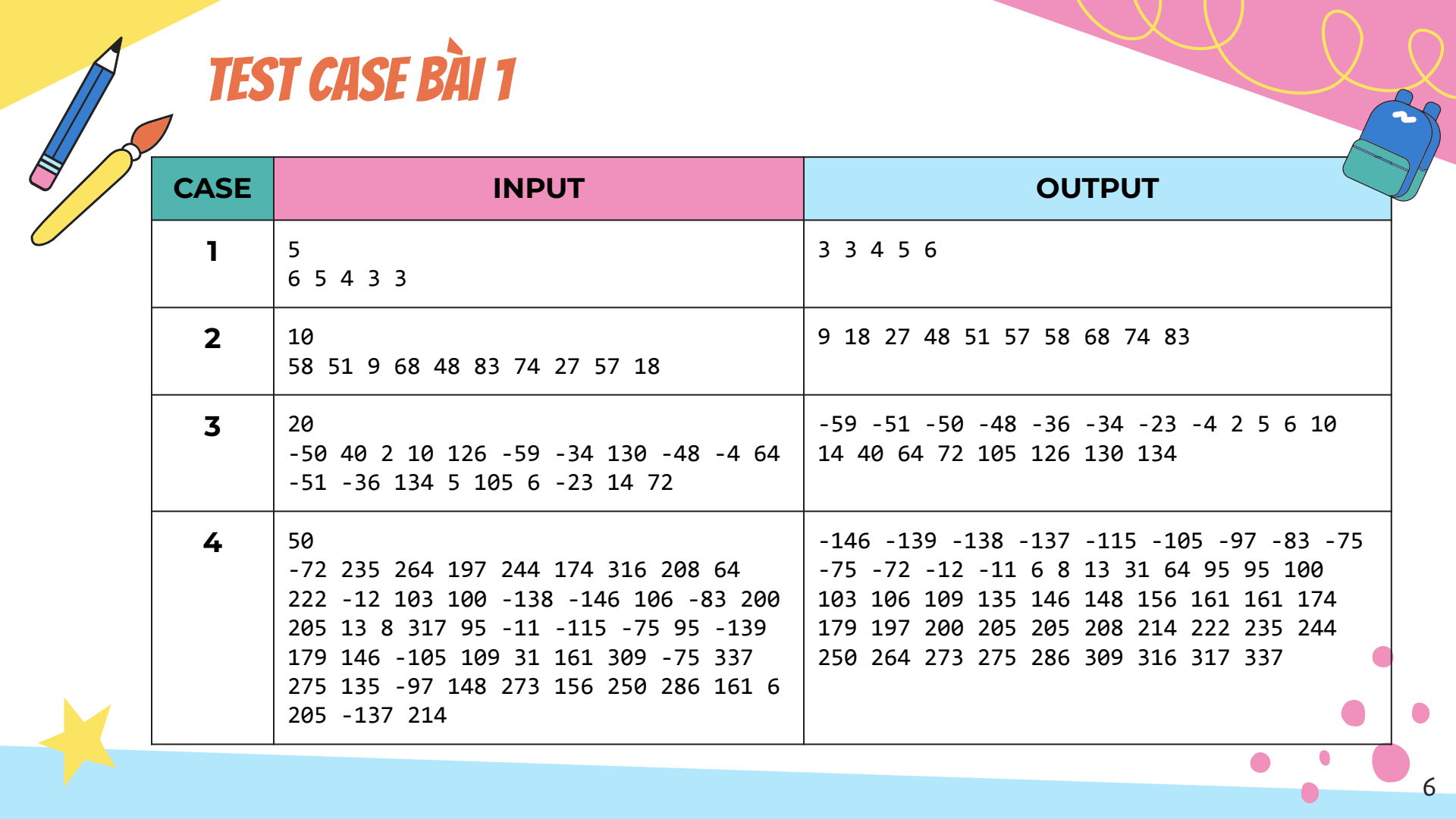
1. Xây dựng hàm `void bubbleSort(int array[], int n)` để sắp xếp một mảng số nguyên `array` có kích thước là `n` theo thứ tự tăng dần.
2. Viết chương trình C nhập các mảng từ bàn phím để kiểm tra tính hiệu quả của hàm `bubbleSort`.
3. Cải tiến đầu vào của chương trình trên bằng một mảng `array` có kích thước là `n` với giá trị phần tử được sinh ngẫu nhiên. Hãy dùng hàm `bubbleSort` để sắp xếp mảng đó theo thứ tự tăng dần.

INPUT:

- Dòng đầu tiên là kích thước mảng `n`
- Dòng thứ hai là mảng `array`.

OUTPUT: Mảng `array` đã được sắp xếp theo thứ tự tăng dần.

TEST CASE BÀI 1



CASE	INPUT	OUTPUT
1	5 6 5 4 3 3	3 3 4 5 6
2	10 58 51 9 68 48 83 74 27 57 18	9 18 27 48 51 57 58 68 74 83
3	20 -50 40 2 10 126 -59 -34 130 -48 -4 64 -51 -36 134 5 105 6 -23 14 72	-59 -51 -50 -48 -36 -34 -23 -4 2 5 6 10 14 40 64 72 105 126 130 134
4	50 -72 235 264 197 244 174 316 208 64 222 -12 103 100 -138 -146 106 -83 200 205 13 8 317 95 -11 -115 -75 95 -139 179 146 -105 109 31 161 309 -75 337 275 135 -97 148 273 156 250 286 161 6 205 -137 214	-146 -139 -138 -137 -115 -105 -97 -83 -75 -75 -72 -12 -11 6 8 13 31 64 95 95 100 103 106 109 135 146 148 156 161 161 174 179 197 200 205 205 208 214 222 235 244 250 264 273 275 286 309 316 317 337

BÀI 2:

YÊU CẦU:

1. Sử dụng chương trình của bài 1, cải tiến hàm **bubbleSort**, thêm biến đếm số lần hoán đổi vị trí của hai phần tử trong mảng **swaps** và biến đếm số lần so sánh hai phần tử trong mảng **comps**.

```
void bubbleSort(int array[], int n, int* comps, int* swaps)
```

Thao tác trên các mảng được sinh ngẫu nhiên.

2. Thực hiện sinh mảng ngẫu nhiên với $n = 10$ và sử dụng hàm **bubbleSort** ở bài 2.1 trên để lập bảng số liệu và tính **số bước trung bình** sau **k lần** thực nghiệm ($k = 100, 1000, 10000$). Mẫu bảng số liệu:

Observation	1	2	3	4	5	...
comps	...	...	...	...	...	...
swaps	...	...	...	...	...	...



03

BÀI TẬP VỀ NHÀ

Một số bài tập vận dụng cao



BÀI 1:

Yêu cầu: Thực hiện các yêu cầu sau sử dụng thuật toán bubble sort để sắp xếp.

- 1.1 Viết chương trình C sắp xếp một mảng số thực nhập từ bàn phím.
- 1.2 Viết chương trình C sắp xếp một chuỗi gồm các chữ cái ngẫu nhiên lấy từ bảng chữ cái alphabet (có bao gồm chữ in hoa).
- 1.3 Cho một mảng a có kích thước n mà mỗi phần tử của mảng đó là một bộ ba số thực. Với $i, j = 0, 1, \dots, n$ bất kỳ, ta định nghĩa:

$$a[i] \leq a[j] \Leftrightarrow F(a[i]) \leq F(a[j])$$

và nếu $a[i] = (a, b, c)$ thì $F(a[i]) = a - 2b + 3c$ với a, b, c là 3 số thực.

Hãy dựa trên định nghĩa quan hệ thứ tự trên, sắp xếp mảng a theo thứ tự tăng dần.



BÀI 2: (SIMILARITY)

Cho một mảng số nguyên `arr` có kích thước `n` và tồn tại ít nhất hai phần tử trùng nhau. Để xóa đi các phần tử trùng nhau, ta có 2 phương án:

1. Duyệt mảng, với mỗi phần tử được xét, ta tìm tất cả phần tử trùng với nó bằng thuật toán `linearSearch` rồi lập tức xóa các phần tử trùng vừa tìm được.
2. Sắp xếp mảng theo thứ tự tăng dần bằng thuật toán `bubbleSort` rồi xóa đi tất cả phần tử trùng nhau.

Sau khi thực hiện 1 trong 2 phương án trên, ta có được mảng `arr` không có phần tử nào trùng nhau.

YÊU CẦU: Em hãy viết chương trình C chạy song song cả hai phương án trên và dùng số bước `steps` có được từ phương pháp thống kê để đánh giá phương án nào là tốt hơn. Vì sao?

